

Exact and heuristic algorithms for the parallel machine total completion time scheduling problem with dual resources, ready times, and sequence-dependent setup times

L. Munoz^a, J.R. Villalobos^a, J.W. Fowler^{b,*}

^a Industrial Engineering School of Computing and Augmented Intelligence, Arizona State University, 699 S Mill Ave, Tempe, AZ 85281, USA

^b Motorola Professor of International Business Dept. of Supply Chain Management, Arizona State University, 300 E. Lemon St., BA 401, P.O. Box 874706, USA

ARTICLE INFO

Keywords:

Parallel Machine Scheduling
Setups
Dual resource
Integer Programming
Photolithography

ABSTRACT

This paper considers a dual resource constrained scheduling problem prevalent in high-capital cost manufacturing industry such as the photolithography area in the semiconductor industry. Specifically, the problem consists of assigning jobs to parallel machines that use common and constrained auxiliary resources such as lithography lens. Very often these auxiliary resources require heavy capital investment and setup times further complicates an already complex dual resource constrained scheduling problem. This paper advocates for the use of deterministic scheduling theory for the design and development of more efficient scheduling strategies. An Integer Programming (IP) model, a heuristic and a hybrid model were developed to schedule identical parallel machines with shared, constrained, auxiliary resources, with sequence-dependent setups, and job release dates with the goal of minimizing the sum of completion times.

An IP model primed with a heuristic can add value to practitioners looking to solve real world resource constrained parallel machine scheduling problems with sequence-dependent setups. It is extremely important to control problem instance size and time horizon in order to obtain near-optimal or efficient solutions within acceptable run times. The proposed heuristic provides an initial feasible solution for the IP model which reduces the space search by decreasing the time horizon and eliminates the time needed to find an initial feasible solution. A tighter formulation is also proposed by reducing the time horizon formulation.

1. Introduction

When production capacity becomes the lead constraint in revenue and profit generation for a manufacturing operation, efficient management of the operations of the production systems can make the difference between a thriving and a failing enterprise. This is particularly true when expanding capacity is limited by the equipment lead time or the high cost of acquiring additional manufacturing resources. Thus, increasing the utilization of existing capacity is extremely important, not only for the manufacturing industry, but also for the economies of the communities in which they operate. One of the industries that can benefit significantly from efficient and effective manufacturing operations is the electronics industry, specifically the semiconductor industry.

According to (Hunter et al., 2002) semiconductor manufacturing fabrication lines are considered to be some of the most complex manufacturing processes in the world. This is due to multiple factors

such as: highly re-entrant and long process flows, significant setup times, highly unreliable equipment, rework, and variable yields. Current methods to schedule and dispatch operations in wafer fabrication lines are far from perfect (Ham, 2018). Thus, there is an urgent need for semiconductor manufacturers to seek efficient and effective methods to better manage their highly complex manufacturing systems in order to improve equipment utilization and meet their financial objectives.

In this paper, we advocate for the use of deterministic scheduling theory for the improvement of the design and development of more efficient scheduling strategies as alternatives to current methods to reduce cycle time in the system. “Sequencing and scheduling are forms of decision-making that play crucial roles in manufacturing and service industries. In the current competitive environment effective sequencing and scheduling has become a necessity for survival in the market-place” (Pinedo, 2016). Scheduling optimization is an approach widely used in manufacturing management to improve the performance of a system by

* Corresponding author.

E-mail addresses: lmunozes@asu.edu (L. Munoz), rene.villalobos@asu.edu (J.R. Villalobos), john.fowler@asu.edu (J.W. Fowler).

optimizing multiple competing objectives such as minimizing machine idle time, setups, job completion times and maximizing throughput and machine utilization. According Hopp and Spearman (2011), increasing throughput of the factory constraint will increase the overall factory throughput to that given by the rate of the new constraint. It is worth highlighting that these complex manufacturing systems not only arise in the electronics and semiconductor industry, but also in industry segments as varied as the telecommunications and plastic injection molding industries, among others. In the semiconductor industry, photolithography is considered one of the factory constraints and represents a complex manufacturing system that can benefit from this study.

The focus of this paper is to develop efficient models for a common and complex situation, i.e., the scheduling of identical parallel machines with shared, constrained, auxiliary resources with sequence-dependent setups and job release dates. We aim to develop linear integer programming (IP) models capable of providing optimal solutions for small and medium size problem instances that resemble those used in real-life practical manufacturing systems. We also aim to develop a heuristic that can help improve the performance of an IP model in those instances where finding an optimal solution is not feasible or practical. The proposed objective function is to minimize total completion time $[\sum C_j]$ using a time-indexed IP model with m parallel machines, r auxiliary resources and j jobs including resource sequence dependent setups $Pm|s_{jk}, rt_j, aux| \sum C_j$. Minimizing $[\sum C_j]$ is a surrogate for minimizing cycle time. Minimizing cycle time is important in the semiconductor industry because it allows manufacturers to identify quality problems quicker and reduces exposure to contaminants; hence, total completion time is proposed to minimize cycle time. Additionally using total completion time as an objective has been common practice in the semiconductor industry, particularly for firms producing microprocessors and memory chips. Ham (2018) solved the dual resource constrained lithography problem using a single objective to minimize the completion time of the last completing machine which attempts to solve a similar problem.

The time-index formulation—originally proposed by (Sousa and Wolsey, 1992)—consists of a time horizon discretized into time periods $1, \dots, H$, where H is an upper bound of the last jobs' completion time (i.e. makespan). The binary variable $x_{jmr} = 1$ if job j is assigned to machine m , resource r to start processing at time t ; otherwise $x_{jmr} = 0$. According to Lawler et al., (1993) if release dates are specified, $1|r_j| \sum C_j$ is strong NP-Hard. Hence, the solution of the proposed time-indexed IP model is NP-hard.

The time-indexed IP model was selected based on a preliminary single-machine scheduling study carried out by Keha et al. (2009). In this paper, the computational performance of four different mixed integer programming (MIP) formulations for various single machine scheduling problems was studied. The results showed that certain MIP formulations that are less frequently used outperformed those commonly used MIP formulations. Ham (2018) also proposed the time-indexed formulation as an alternative to his mixed-integer programming (MIP) "positional & assignment variable" model applied for the dual resource constrained lithography production problem. An overview of mixed IP and MIP formulations for manufacturing systems and lithography research is presented next.

2. Literature review

In this section of the paper we present previous works related to the problem at hand. Several comprehensive surveys on scheduling problems that consider setups have been published since 1979. A recent comprehensive survey by Allahverdi (2015) lists hundreds of papers and suggests the need for models that mimic real-world practical problems Section 2.1 focuses on photolithography scheduling and section 2.2 presents Mixed-integer Linear Programming (MIP) and integer programming (IP) models for single and parallel machine scheduling problems with setup times.

2.1. Photolithography Machine scheduling

Photolithography—the process step in semiconductor manufacturing in which circuit patterns are transferred onto the wafer through a lens known as a reticle—is a complex manufacturing process due to its highly reentrant nature. It is necessary to subject the wafer under fabrication to several applications of lithography to get the final version of the edged circuitry, leading to the reentrant nature of the semiconductor manufacturing process.

"Photolithography is usually the bottleneck process with the most expensive equipment in a wafer fab. Being one of the processes that is repeated the most during fabrication, any reduction in photolithography cycle time will reduce overall fab cycle-time" (Akcalt et al., 2001). A MIP model using a 2-phase approach to machine scheduling in lithography by integrating dispatching logic with a MIP technique has been proposed (Ham and Cho, 2015). For phase 1, the mathematical model aimed to minimize 1) auxiliary resource setup cost, 2) total completion times, 3) output target over-achievement and 4) output target under-achievement. For phase 2, a heuristic aiming to optimize job dispatch sequence for each machine was used. The heuristic iterated through the entire job set, solving an MIP assignment problem and removing the selected job from the set and adding it to the schedule (i.e. Gantt chart). This process was repeated until the job set was empty which means all jobs were scheduled. Ham (2018) published the "Scheduling of Dual Resource Constrained Lithography Production" model to sequence job, machines, and auxiliary resources. This problem becomes more challenging when auxiliary resources must be tracked, and a setup time is incurred as a function of auxiliary resource location. For example, an auxiliary resource-sequence-dependent setup takes one to five units of time and the resource transfer time between machines takes 10 to 34 units of time. His conclusion, based on the results obtained, was that constraint programming (CP) is a powerful method that can be used to solve dual resource constraint problems obtaining optimal results in shorter times. That is, CP is a promising method that reached optimality criteria for industry-size problems (200 jobs) for a single objective C_{max} . The model found optimal solutions within two minutes and a Gap of less than 1% Gap against the best solution found (Ham, 2018).

A similar IP model based on an objective to meet daily production targets while reducing the number of auxiliary resource setups was proposed by Yan et al. (2011). A branch-and-cut method was applied to solve the problem. The authors in this paper do not distinguish among different jobs and mainly focus on machine and auxiliary resource setups which differs from Ham and Cho's approach. Yan et al. (2012) modified their previous model to include load balancing and auxiliary resource expiration. The authors applied IP models with capacity constraints and process requirements. The objective is to meet daily targets while keeping the load balanced to minimize auxiliary resource shortage and machine setups. Yan et al. (2013) proposed a new lithography machine scheduling solution approach based on convex hull analyses. Since the convex hull of the problem was difficult to solve, a two-phase approach was utilized in order to solve the problem within reasonable time. Phase 1 consisted of removing certain problematic constraints (i.e. auxiliary resource setups) from the original full-size problem in order to solve it more efficiently. Then, phase 2 efficiently solves the problem with the full set of constraints within a reduced decision space. That is, the problem was simplified by fixing the machine-auxiliary and resource-layers assignment and/or fixing the number of jobs scheduled in each machine-auxiliary resource combination.

Díaz et al. (2005) conducted an evaluation of the impacts of reticle requirements in semiconductor wafer fabrication. A discrete-event simulation model of the photolithography station was created and coupled with a network flow optimization model in order to optimize reticle location. The results showed that "ready" and ready-slack" rules outperformed several other rules with respect to layer outs and total weighted tardiness.

Shahvari and Logendran (2017) studied a bi-objective batch

processing problem with dual-resources on unrelated-parallel machines to mimic an environment with machines and operators. Dual-resources are required to process the setup and run of any batch on any machine. Particle swarm optimization (PSO) is proposed to solve medium and large problem instances. This paper is similar in some aspects to the problem being explored, but there are key differences. For instance, the [Shahvari and Logendran \(2017\)](#) model is well suited for diffusion tools in the semiconductor industry where multiple jobs get batched together. Our paper is suited for photolithography machines which process a single job at a time and the objective is to maximize throughput while minimizing auxiliary resource travel time between machines and setup time at the machine. The objective of [Shahvari and Logendran \(2017\)](#) is to find the final pareto frontier using MILP and *meta*-heuristics such as PSO and “Technique for order preference by similarity to ideal solution (TOPSIS)”. The solutions found by MILP and heuristics were evaluated using four different metrics: a) pareto dominance indicator to measure the ratio between the non-dominated solutions contributed by search algorithm j and all the non-dominated solutions, b) distribution diversity indicator which measures the extend of the spread achieved among the solutions by algorithm j , c) spacing indicator which measures the spread of the achieved non-dominated set of j^{th} algorithm and d) convergence indicator which measures the solutions’ convergence to the origin of coordinates. [Shahvari and Logendran \(2017\)](#) concluded that the unrelated-parallel batch processing problem with dual resources is a complex problem with broad applications in manufacturing industries with batch processing machines. The optimization method—with the help of “Technique for order preference by similarity to ideal solution (TOPSIS)” —provided the highest rank between the search algorithms for small problem instances while the hybrid multi-objective PSO provided the highest rank for medium/large problem instances. On the other hand, this paper aims to find near optimal solutions for medium and large problem instances with a single objective in the least amount of time for lithography scheduling systems.

In summary, photolithography scheduling is available in the open literature but perhaps limited. Our findings focused on MIP, IP and CP formulations to solve the complex photolithography system. Prior to analyzing single and multiple parallel machine scheduling literature we must note that none of the publications applied the time-indexed IP method proposed in this study.

2.2. Single and parallel machine scheduling using IP/MIP formulations

In this section we present previous works on single machine ([Section 2.2.1](#)) and parallel machine scheduling formulations ([Section 2.2.2](#)). We highlight exact scheduling methods that have been studied for many years in order to find optimal schedules.

2.2.1. Single machine scheduling

This subsection provides a brief overview on five key formulations. Most of the research conducted on IP scheduling models have emanated from the following formulations.

1. Manne formulation - completion time is modeled as continuous variable ([Manne, 1960](#))
2. Potts formulation - job sequence is modeled by binary linear ordering variables ([Potts, 1980](#))
3. Wagner formulation - variables are used to represent a fixed number of slots per machine ([Wagner, 1959](#))
4. Sousa and Wolsey formulation - discrete time periods for planning horizon are used ([Sousa and Wolsey, 1992](#))
5. Pessoa et al. formulation - discrete time periods for horizon and precedence relationships are introduced ([Pessoa et al., 2010](#))

[Table 2-1](#) summarizes previous publications based on IP as cited by [Nogueira et al. \(2019\)](#). This table summarizes papers by formulation and denotes if ready times for job j (rt_j) and setup times between job i and j

Table 2-1

IP publications for single machine scheduling ([Nogueira et al., 2019](#)).

MIP Formulations	rt_j & s_{ij} considered?	$\sum w_j C_j$
Manne	None considered rt_j only s_{ij} only	Keha et al. (2009), Queyranne and Wang (1991) Keha et al. (2009) Queyranne (1993)
Potts	None considered rt_j only	Blazewicz et al. (1991) Chudak and Hochbaum (1999) , Keha et al. (2009) Dyer and Wosley (1990) , Keha et al. (2009), Queyranne et al. (1994) , Unlu and Mason (2010)
Wagner	None considered rt_j only	Keha et al. (2009), Khowala et al. (2005) Lasserre and Queyranne (1992) , Queyranne et al. (1994) Keha et al. (2009)
Sousa and Wolsey	None considered rt_j only	Keha et al. (2009) Khowala et al. (2005) Avella et al. (2005) Keha et al. (2009), Queyranne et al. (1994)
Pessoa et al.	None considered	Abeledo et al. (2010) , Battarra et al. (2013) , Bigras et al. (2008)

(s_{ij}) were investigated in the papers listed.

According to [Nogueira et al., \(2019\)](#) the number of jobs (n) and the length of the time horizon (h) significantly impact the time-index formulation which presents a polynomial number of constraints and variables with $O(nh)$ variables and $O(n^2h)$ constraints. “It is worth noting that as the horizon (h) $\gg n$ jobs, $h \propto n$, “[Pessoa et al.](#)”, “[Sousa and Wolsey](#)” and “[Sousa and Wolsey Improvement](#)” formulations will increase in size faster than other formulations“ ([Nogueira et al., 2019](#)). Even though the papers in [Table 2-1](#) considered total weighted completion time, the conclusions and recommendations were valuable in our research.

2.2.2. Parallel Machine scheduling with setup times

This subsection deals with m parallel machines and machine setups that are dependent on the job sequence. That is, setup times depend on the outgoing job i and incoming job j . According to ([Allahverdi, 2015](#)) multiple scholars have conducted research on non-family scheduling with setup times, the different methodologies are depicted in [Table 2-2](#). A family is defined as a set of jobs with similar characteristics in terms of setups, tooling and operation sequence. The number of job families is fixed as well as the number of jobs in each family. A setup on a machine is incurred when a job from a different family is to be processed. There is no setup time/cost to change from one job to another within the same family. A major setup time/cost is incurred between job families. Non-

Table 2-2

Non-family scheduling with setup times (From [Allahverdi, 2015](#)).

Authors	Method Applied
Fan and Tang, (2006)	Integer programming model, a column generation algorithm for parallel machines
Hou and Guo, (2013)	MIP with Genetic Algorithms. Multiple resource constraints
Hu and Yao, (2010)	MIP model, lower bound, GA
Rocha et al., (2008)	MIP, B&B algorithm.
Toksari and Güner, (2010)	MIP

family scheduling is defined as a set of jobs with different characteristics and a setup on a machine is incurred between any job (Allahverdi, 2015).

According to (Behnamian et al., 2009), most of the existing publications use an IP model for small size problem instances and a heuristic or metaheuristic for larger size problem instances. Machine scheduling has received a significant amount of attention over the last few decades with over 1,000 research papers published, including three comprehensive reviews Allahverdi et al., (1999); Allahverdi et al., (2008); and Allahverdi, (2015). Most scheduling environments involve setup operations in the real world; however, only ~10% of the published research in scheduling includes setup times or setup costs. Hence, to address industry needs, more research on scheduling problems with explicit consideration of setup times/costs is needed (Allahverdi, 2015).

Prior to introducing the methodology for this study, we must emphasize that there is a need to build scheduling models that better reflect the reality of the factory floor so that practitioners gain insight to handle the complex problems they face day to day, that is the underlying motivation of this work. We present next the proposed methodology to address the problem at hand using exact and heuristic algorithms.

3. Problem definition and methodology

The manufacturing problem we are addressing considers jobs that must be processed by one machine and one auxiliary resource simultaneously. Also, we assume that the auxiliary resource cannot be assigned to more than one machine at the same time. Auxiliary resources are discrete, non-renewable and constrained since they need to be transferred and transferred between machines to process the job. The environment has a set of jobs J either incoming to the system ("ready times") or available for immediate scheduling. These jobs can be processed in one or more of identical parallel machines if fitted with one of the auxiliary resources r in set R . These scarce resources are shared among some or all machines. Incoming jobs require a release date to prevent assignment before they become available. "Once a job is assigned to a machine and an auxiliary resource, that job must complete its processing with the machine-auxiliary resource assigned. That is, preemption is not allowed. The allocation of a job to a machine-auxiliary resource may imply the addition of a setup time. The interaction among jobs, machines and auxiliary resources can be described as a job loaded in a machine with a lens or reticle (auxiliary resource) needed to process the job. A setup time is incurred between jobs of different families. Thus, the machine must temporarily idle for the auxiliary resource setup. If jobs belong to the same family, no setup is required. In practice, a family consists of a combination of specific product and an operation; however, for this paper a family is created by randomly assigning jobs to family sets forcing each set to have a similar number of jobs. If a resource is transferred from one machine to another one, an additional setup time (travel time between machines) is required. Inefficient resource

assignments may cause a machine to be temporarily idle until the proper auxiliary resource is installed. Typically, auxiliary resources are finite and expensive that need to be shared among different machines, thus if a resource is being used in one machine, another one may be idle waiting for this resource. Thus, generating efficient schedules not only optimizes resource assignment and utilization, but also overall factory throughput resulting in lower cycle times.

Figure 3-1 depicts the possible combinations among jobs, machines and auxiliary resources. Incoming jobs denote jobs that are not at the workstation yet. That is, incoming jobs are processing at a different workstation and can be scheduled until they arrive at the workstation (parallel machines). The following methodology is proposed to build solutions for the scheduling problem found in manufacturing operations as the one outlined in Fig 3-1:

1. Capture the scheduling problem using IP models
2. Find optimal solutions for reduced-size instances using the IP models,
3. Based on results of step2, build heuristics algorithms to find fast feasible solutions for larger size instances
4. Use the solutions rendered by the heuristics in step 4 to prime time-indexed IP models as a starting solution (hybrid model approach)
5. Use the best solution in step 4 under constrained computational time as the best candidate for implementation.

The objective of using this methodology is to attempt to improve the solution found by the heuristic using branch and bound algorithms provided by the IP solver. One of the goals of this study is to find near-optimal solutions for the manufacturing system previously described by applying time-limited exact optimization methods, such as the time-indexed IP presented by Sousa and Wolsey (1992). The second goal is to develop a heuristic capable of generating fast and feasible initial solutions when the IP approach is not capable of finding one. The third goal is to generate a hybrid model with the combination of the IP model and the proposed heuristic. That is, the solution found by the heuristic is given to the IP model aiming to reduce the time horizon and provide an initial solution to the IP model. We believe a combination of an exact and a heuristic algorithm would allow practitioners to tackle medium and large size problem instances where setups are required for machine and auxiliary resources in complex manufacturing systems. It is worth noting that computational complexity is based on the process times for the activities since we use a time-indexed formulation. Other formulations that do not use time as index are not affected by process times. It is worthwhile to mention that setup between job j and i was modified to setup between resources providing the same results and smaller setup matrices are obtained. We get the same results since jobs belong to families and auxiliary resources are unique for each family.

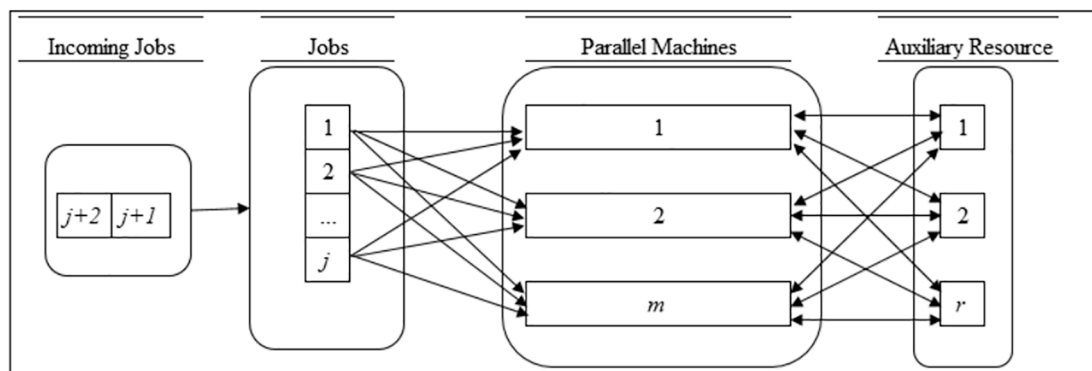


Fig. 3-1. Manufacturing System with Parallel Machines and Auxiliary Resources.

4. Integer programming mathematical model

We now turn to defining the time-indexed IP model including the indices, sets, objective function and constraints.

Indices and Sets.

Define:

j index for jobs, $j \in J$

r index for auxiliary resources, $r \in R$

m index for machines, $m \in M$

t index for time, $t \in |H|$

H set of periods (total processing time plus maximum ready time) where $|H| \geq \sum_{j \in J} p_j + \max\{rt_j\}$

H' set of periods (total processing time plus maximum ready time) where $|H'| \geq \Theta * \left(\frac{\sum_{j \in J} p_j + \max\{rt_j\}}{m} \right)$ where $2 \leq \Theta \leq m$ for tighter formulation.

H should be used for single machine scheduling or when model parameters are not well understood. Otherwise, use $|H|$ formulation and a machine factor (Θ) of 2 or greater to tighten the formulation. This paper uses $\Theta = 2$ for 45 problem instances without observing any infeasibility issues.

Parameters:

rt_j ready time for job j

p_j processtime for job j

$s_{r',r}$ setup time between resource r' and resource r

$tt_{m',m}$ auxiliary resource transfer time between machine m' and machine m

Decision variables:

$$x_{jmt} = \begin{cases} 1 & \text{if job } j \text{ is assigned to machine } m, \text{ resource } r \text{ to start processing at time } t; \\ 0 & \text{otherwise} \end{cases}$$

Formulation.

The objective function aims to minimize the total sum of completion times (C_j). That is, if $x_{jmt} = 1$, the process time for job j is added to start time t which determines the completion time of job j . It is noteworthy that each job-machine combination has a specific auxiliary resource, but these resources are shared among multiple jobs-machines according to the density matrix.

$$\min \sum_{j \in J} C_j \quad (1)$$

Subject To:

$$\sum_{m \in M} \sum_{r \in R} \sum_{t=rt_j}^{H'-p_j+1} (t + p_j - 1) x_{jmt} \leq C_j \quad \forall j \in J \quad (2)$$

$$\sum_{m \in M} \sum_{r \in R} \sum_{t=rt_j}^{H'-p_j+1} x_{jmt} = 1 \quad \forall j \in J \quad (3)$$

$$x_{jmt} + \sum_{r' \in R} \sum_{i=\max(rt_j, t-p_i-t_{m',m}-s_{r',r}+1)}^{\min(t+p_j+t_{m,m'}+s_{r,r}-1, H'-p_j+1)} x_{im'r'i} \leq 1$$

$$\forall i, j \in J : i > j; m, m' \in M : m = m'; r \in R; t \in \{rt_j \dots H' - p_j + 1\} \quad (4)$$

$$x_{jmt} + \sum_{m' \in M} \sum_{i=\max(rt_j, t-p_i-t_{m',m}-s_{r',r}+1)}^{\min(t+p_j+t_{m,m'}+s_{r,r}-1, H'-p_j+1)} x_{im'r'i} \leq 1$$

$$\forall i, j \in J : i > j; m \in M; r, r' \in R : r = r'; t \in \{rt_j \dots H' - p_j + 1\} \quad (5)$$

$$x_{jmt} \in \{0, 1\}, C_j \in \mathbb{Z}^+ \quad (6)$$

The first constraint set (eq. (2)) calculates the completion time for each job j , the expression in parentheses serves to offset the process time for each job. The second constraint set (eq. (3)) forces all jobs to start processing at only one period t between the ready time and the latest time it can start (and still finish by the end of the horizon) per machine-resource combination. The third constraint set (eq. (4)) prevents job-machine-period overlap and incorporates two different setups: i) resource-sequence dependent setup and ii) a machine-sequence dependent setup which is incorporated in the form of resource travel time between machines. It is worth pausing for a moment to highlight the resulting computational complexity of constraint (eq. (4)), since all combinations between job j and i where $i \neq j$ must be explored. All combinations between machine m and m' where $m = m'$, also need to be explored. Similarly, all resource combinations must be explored between r and r' where $r = r'$. We estimate the model will generate $O(jmrt)$ variables and $O(jmt^2r)$ constraints. The constraint sets (4) and (5)

generate the most constraints in this formulation, up to 6.8 million among all the constraints in our experimentation.

Similarly, constraint set (eq. (5)) enforces that at any given time t at most one job can be processed on a given machine and resource respectively. This formulation resembles real-world complex manufacturing environments in the semiconductor industry, mold injection operations in the PCB industry. Equation (6) defines a binary decision variable x and a non-negative integer variable for completion time.

5. Heuristic for $Pm|s_{jk}, rt_j, aux| \sum C_j$

This paper proposes a heuristic to find fast feasible solutions that can be used to prime the proposed IP model with an initial feasible solution. The proposed heuristic aims to minimize weighted completion time by

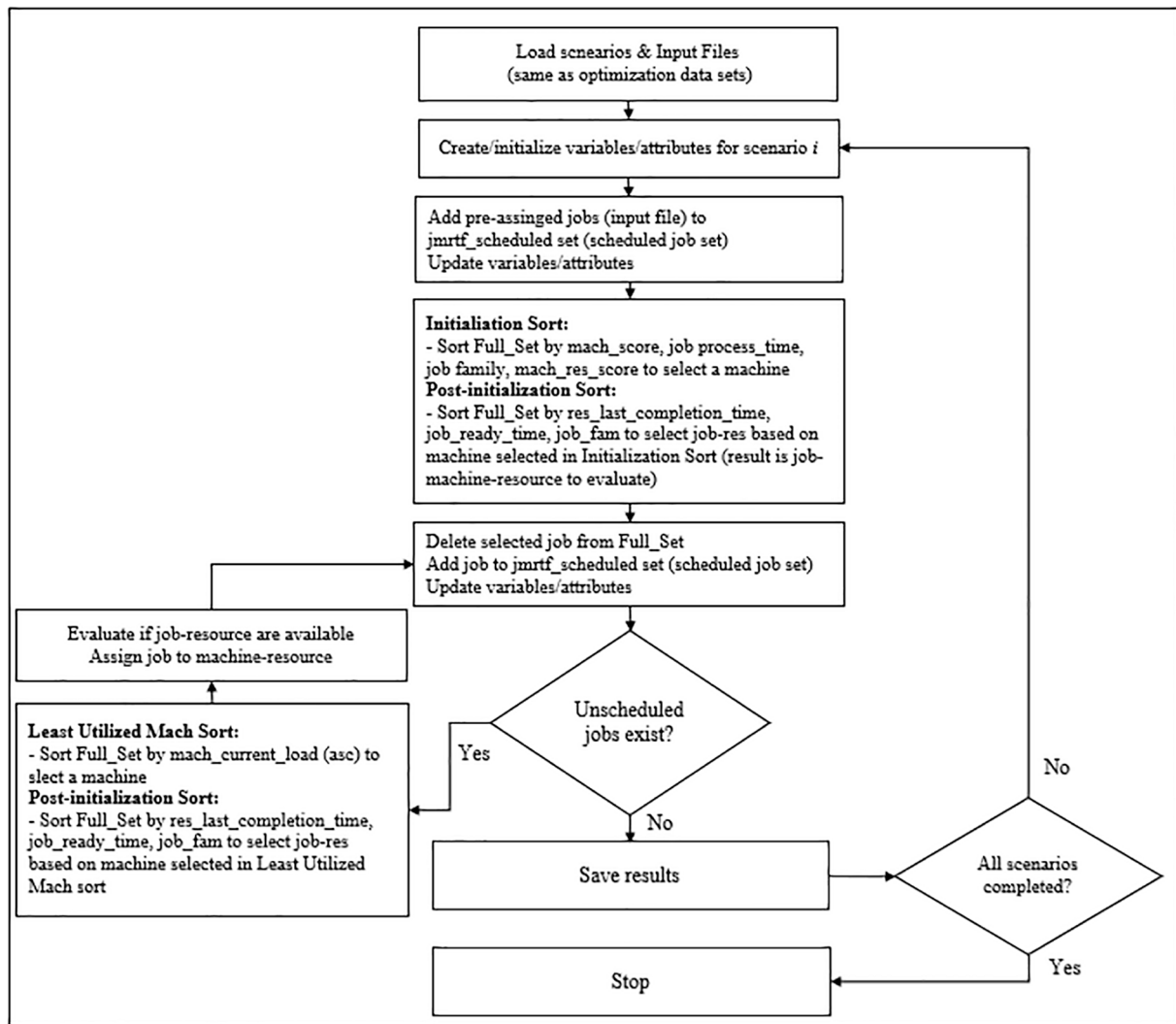


Fig. 5-1. Heuristic Process Flow.

reducing auxiliary resource setups. The objective is accomplished by scheduling as many jobs as possible of the same family to the same machine and resource. That is, all jobs that can run on the same machine/resources and do not require a setup (i.e. jobs are compatible due to same resource type) are said to be part of the same family. Jobs in the same family are scheduled and/or sequenced together in order to reduce setups and minimize auxiliary resource travel time when moved between machines when possible. That is, ready times may not allow all jobs of the same family to be scheduled back to back.

The heuristic aims to reduce completion time and accomplishes its objective through a series of sorts and assignments as depicted in Fig. 5-1: i) a set of pre-assigned jobs randomly selected (i.e. set name jmrt_pre-assigned) are given to the model to simulate that all machines start with a pre-assigned job in the busy state. ii) then, the pre-assigned jobs are removed from the “Full_Set” which contains all unscheduled jobs in addition to other attributes (See Figure 5-3) to run the heuristic. The pre-assigned jobs are added to the “jmrtf_scheduled” set which contains all jobs that have been scheduled including the following attributes: job id, machine id, resource id, family id, process time, ready time, start time and end time. Model variables and attributes are updated to reflect the pre-assigned jobs. iii) An initialization sort logic consisting of four attributes (i.e. mach_score, job process_time, job family and mach_res_score) is executed. The four attributes are sorted ascending or descending in order to select the machine to be loaded next. It is worth noting that four attributes sorted in two different ways (ascending or

set jmrf	Set jmrt pre-assigned	set job
j,m,r,f	j,m,r,t	j,f,p,t,r,t
1,1,1,1	1,1,1,5	1,1,8,5
1,2,1,1	10,2,3,7	...
...		10,2,16,7,1
10,2,4,2		

set m2m Time	set r2r Time
m1,m2,Time	r1,r2,Time
1,1,0	1,1,0
1,2,7	1,2,7
2,1,7	2,1,7
2,2,0	2,2,0

Fig. 5-2. Input Sets.

descending), similar to the levels of a factorial experiment design resulting in the generation of 16 combinations used to implement the heuristic. Then, a post-initialization sort is executed by sorting 3 attributes (i.e. `res_last_completion_time`, `job_ready_time`, `job_fam`) in either all ascending or all descending resulting in two additional combinations. Hence, a total of 32 sorting variations are executed for each instance based on the pre-initialization and post-initialization logic. These combinations are depicted in Table 6-3 by attribute name in the *Experimental Design* Section of this paper. iv) After the initialization logic step, all future sorts for the job-machine-resource combinations aim to select the least utilized machine-resource combination. Machine-resource utilization is tracked using the “`mach_current_load`” and “`res_current_load`” attributes that track total time assigned to those resources. The “Least Utilized Mach Sort” logic consists of sorting “`mach_current_load`” attribute ascending to select the least utilized machine as shown in Fig. 5-1. Then, the job-resource are selected and evaluated to make sure they are available. That is, the resource has to be readily available. The process it is repeated until all jobs are assigned and until all the scenarios are generated as outlined in Fig. 5-1.

The full set of attributes for jobs, machines (`mach`) and resources (`res`) that are used to keep track of time variable and the history of jobs assigned to machine and resources is shown in Fig 5-3.

A full set of input files and variables is presented next in Fig 5-2. These input sets are given to the optimization IP model and the heuristic. Set “`jmrf`” represents the job-machine-resource-family combinations and the sample shown represents the job id, machine id, resource id and family id. Similarly, the set “`jmrt pre-assigned`” represents the job-machine-resource-family randomly pre-assigned (part of the input files) where the numbers shown in the figure below represent job id, machine id, resource id and job’s start time. The set “`job`” provides the job-family-process time (pt)- ready times (rt) combinations. Set “`m2m Time`” provides the resource travel time from m1 id to m2 id. The “`Set r2r Time`” provides the setup time when swapping resource r1 id to r2 id. It must be noted that in this study the start time is set at $t = 1$.

The following section provides an explanation on how the attributes are initialized.

- o `mach_score` = sum of process time of all jobs capable of running on a given machine.
- o `mach_last_res` = 0 unless a job-resource was pre-assigned.
- o `mach_current_load` = sum of the process time of the jobs assigned to it.
- o `mach_last_completion_time` = $t_{start} + \text{process time} - 1$ for the last job assigned.
- o `mach_res_score` = 1 for all machine-resource combinations during initialization. Then, set = 0 for the machine-resources combinations that received a pre-assigned job.
- o Sorting by `mach_res_score` (asc) after the job’s first pass allows the heuristic select the next job of the same family aiming to eliminate a setup.

- o `res_last_machine` = 0 unless a job-machine was pre-assigned.
- o `res_current_load` = sum of the process times of the jobs assigned.
- o `res_last_completion_time` = $t_{start} + \text{process time} - 1$ for the last job assigned.

The pseudo code associated with the heuristic can be provided upon request.

The decision variable x_{jmrt} (heuristic output) is generated using the attributes presented in Fig 5-3 which allow the model keep track of all job-machine-resource assignments and time.

A job can be scheduled if it is readily available (i.e. variable `job_avail` = True). That is, the job’s ready time must be equal or less the machine’s last completion time attribute denoted by `mach_last_completion_time` + 1 unit. A resource can be scheduled with a job-machine if the resource is available. That is, the variable `Res_avail` = True if resource is not being utilized by other machine or the resource is not being transferred between machines based on the set `m2m times`.

6. Experimental design

The experimental design adopted in this study aims to represent real-world practical problems. Hence, the following three factors were selected to generate the instances to be part of the experiment: problem instance size, problem instance complexity and job-machine sharing density (also known as machine eligibility restrictions in the parallel machine scheduling literature). Problem instance size is defined as a function of the number of jobs, machines and auxiliary resources. For instance, the small problem instance of 10|2|4 is defined as a combination of 10 jobs, 2 machines and 4 auxiliary resources.

We define problem instance complexity as a function of process times, release dates and resource setup/delivery times (cf. Sousa and Wolsey, 1992).

We define sharing density as the percent of job-machine combinations (i.e. 100%, 66% and 33%) that are viable for each scenario described in Table 6-1 Based on personal experience, real-world problem instances resemble more of a sparse matrix than a dense one.

A total of 45 scenarios resulted by combining 3 factors (size, complexity and sharing density) at 5, 3, 3 levels respectively. We introduced three types of model complexities: reduced, medium, and

Table 6-1
Design of experiments.

Instance Size Jobs Mach Res (number families)	Instance complexity	Sharing density
• 10 2 4 (2 families)	• Reduced	• 33%
• 25 3 6 (3 families)	• Moderate	• 66%
• 50 5 12 (6 families)	• Complex	• 100%
• 75 7 18 (9 families)		
• 100 10 24 (12 families)		

Full_Set (Array with all needed fields and attributes to keep track of assignments and allow quick sorting)

job-machine-resource-family-process_time-ready_time columns from input sets

machine attributes: `mach_score` = SumOfProcTimes | `mach_last_completion_time`=0 | `mach_last_fam`=0 | `mach_last_res`=0* | `mach_current_load`

resource attributes: `res_current_load`=0 | `res_last_completion_time`=0 | `res_last_fam`=0 | `res_last_mach`=0

`mach_res_score` = 1 (all combinations initially); then, set =0 for mach-res with jobs pre-assigned

jmrtf_scheduled set (keep track of scheduled jobs, start time, end time)

time | job|mach|res|pt|rt|fam|t_start|t_end

where $t_{start} = \text{mach_last_completion_time} + 1 + \text{total setup}$

$t_{end} = \text{mach_last_completion_time} + 1 + \text{total setup} + \text{process_time}$

where total setup = resource setup + resource delivery time between machines

Fig. 5-3. Heuristic Sets for Scheduled and Unscheduled jobs.

Table 6-2
Complexity design parameters.

Complexity	PT UB	RT UB	RTTBM LB	RTTBM UB
reduced	5	5	1	2
medium	10	10	3	4
hard	20	20	5	9

hard complexity as shown in Table 6-2. Reduced complexity was defined as solving a problem with process times (PT) and ready times (RT) uniformly distributed between 1 and 5 units of time. Setup time is 1 unit and resource travel time between machines (RTTBM) is between 1 and 2 units of time. The process time and ready time lower bound was 1 unit of time, for the upper bounds see Table 6-2 for detailed parameters utilized for the medium and hard complexity cases.

We believe these levels of problem instance size, complexity and sharing density should provide insight to practitioners on the proposed model's capability. Practitioners may adjust real processing times by diving processing time by n units/period to extrapolate results in this paper and/or to apply this model to their practical problems. For instance, if the largest process time of a job is 100 min, a model complexity estimation can be obtained by dividing the time by 5 min/unit period which resembles the complex scenario at 20 units process time. This procedure should allow large practical problem instances to be modeled by the proposed methods based on our experience in the electronics and semiconductor industries.

The heuristic experimental design consisted of running 32 sorting variations for each of the 45 instances resulting in 1440 runs. The lowest completion time was selected to prime the IP model. If there was a tie among different scenarios, the one with the lowest run time was selected. Table 6-3 depicts the 32 sorting variations consisting of 4 attributes at 2 levels each for the initial sort logic and two combinations for the post initialization sort.

Where the sorting was either ascending or descending for each attribute. The post initial sort only tested either all 3 attributes ascending or the 3 attributes descending to limit the number of variations.

The proposed methodology is to run the optimization IP model first for the 45 scenarios. Then, run the heuristic with the same data set comprised of 45 scenarios with 32 sorting variations and with one addition attribute used to identify lots that belong to the same family. The job-machine-resource data set becomes the job-machine-resource-family set for the heuristic to signal what jobs are compatible with each other and do not require resource change. It is worth mentioning that the data sets are identical except for the addition of the family attribute.

The criterion to stop the IP model was one hour of computational time if an optimal solution was not found. The decision to stop the model after one hour is based on our assumption that practitioners in industry may not find a model attractive if it takes one or more hours to provide a solution unless the model is used for strategic decision making. In order to further understand the maximum run time impact and/or behavior on the time indexed IP model, the IP model was run again for three

complexity levels using the J50|M5|R12|D = 0.66 instance at 30, 60, 90 and 120 min maximum run times. According to Ham (2018) practitioners look for solutions under five minutes. The experimental results for the time-indexed IP model are presented next.

7. Experimental results

This section presents the experimental results obtained by the time indexed IP optimization model, the heuristic and the hybrid model which is a combination of the IP model and the heuristic.

All the experiments for this study were performed in a virtual server with Intel® Xeon® CPU E5-2650 0 @ 2.00 GHz (2 processors) with 64 GB RAM, and windows server 2016 operating system. IBM Cplex version 12.9 with default settings was used to solve the models. OPL language was used to prototype the final model which was translated to C Sharp in order to speed up the time to build the models. Python was used to generate the randomized data sets and the proposed heuristic.

7.1. Experimental results IP model For $Pm|S_{jk}, rt_j, aux| \sum C_j$

The computational results for the time-indexed IP model are depicted in Table 7-1 for the 45 problem instances. The first column gives the problem instance as denoted by the combination of complexity, number of jobs-machines-resources and sharing density—Com|Jobs|Mach|Res|Den. The second column gives the objective function value, the third column provides the best bound generated by the solver, the fourth column shows the solution time in seconds. The fifth column gives the Gap (%) provided by Cplex as the difference between the best integer solution and the best bound. The last two columns give the number of variables (Vars) and constraints (Cons) generated by the model. As previously stated, a total of 45 scenarios were run as a result of the combinations among 3 complexity levels, 5 job-machine-resource combinations and 3 job-machine density levels.

Optimal solutions were found for 16 out of the 45 scenarios in less than one hour for several combinations of 10/25 job sizes. A Gap of 10% or less was found for 24 out of the 45 scenarios ranging from 10 jobs up to 75 jobs (i.e. MED|J75|M7|R18|0.33). Optimal solutions were found for 15 out of the 45 scenarios for the small instances (10/25 job) in less than 600 s (10 min). No results are reported for one out of the 45 instances (HAR|J100|M10|R24|1.0) due to constraints equation (4) and (5) making the model excessively large. It must be noted that prior to using H' time horizon formulation the H formulation rendered at least 5 instances out of memory. Hence, the authors decided to tighten the model using the H' formulation described in section 4.

It is evident that there is a positive correlation (factor = 0.68) between Gap % and the number of constraints generated as depicted in Fig 7-1. It is also evident that the relationship is not linear, but rather logarithmic.

The Gap % summary in Table 7-2 shows the IP formulation presents higher Gaps for the instances with 100% sharing which indicates the solution space is larger; hence, the model iterated through more combinations. Table 7-2 also shows that Gaps gets larger as the number of Jobs increases. However, the Gap for the complexity instances did not follow a particular pattern despite the fact that the hard instances have

Table 6-3
Heuristic sorting combinations.

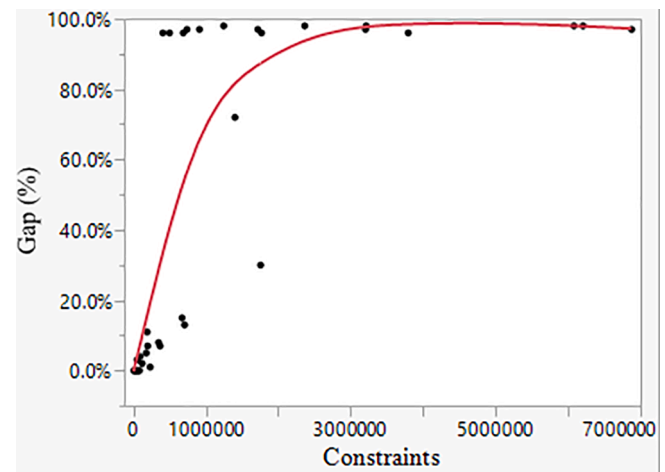
Sort type	Attribute	Levels	Combinations
Initial Sort	mach_score	asc/desc	16
	Job_process_time	asc/desc	
	Job_family	asc/desc	
	Mach_res_score	asc/desc	
Post Initial	res_last_completion_time', 'job_read_time', 'Job_family'	Asc or desc	2
Sort			
Total Heuristic Variations			32

Table 7-1IP Model Results for... $Pm|s_{jk}, r_{tj}, aux| \sum C_j$.

Com Jobs Mach Res Den	Obj Funct	Best Bound	Sol Time (sec)	Gap (%)	Vars	Cons
RED J10 M2 R4 0.33	128	128	0.9	0%	542	1740
RED J10 M2 R4 0.66	110	110	0.3	0%	556	2362
RED J10 M2 R4 1.0	118	118	0.5	0%	756	4207
RED J25 M3 R6 0.33	464	464	46.4	0%	2282	17,521
RED J25 M3 R6 0.66	322	322	43.3	0%	2409	24,466
RED J25 M3 R6 1.0	341	341	693.2	0%	4139	66,275
RED J50 M5 R12 0.33	991	957	3601.1	3%	5086	51,560
RED J50 M5 R12 0.66	937	873	3602.4	7%	10,622	200,233
RED J50 M5 R12 1.0	1660	62	3601.0	96%	15,794	500,112
RED J75 M7 R18 0.33	1451	1292	3601.8	11%	11,891	191,405
RED J75 M7 R18 0.66	2231	85	3601.5	96%	21,361	689,285
RED J75 M7 R18 1.0	2545	88	3602.5	97%	36,290	1,719,167
RED J100 M10 R24 0.33	3072	118	3601.7	96%	19,385	409,063
RED J100 M10 R24 0.66	3087	125	3604.1	96%	40,324	1,769,745
RED J100 M10 R24 1.0	3063	122	3610.3	96%	59,085	3,793,645
MED J10 M2 R4 0.33	232	232	2.5	0%	1002	3235
MED J10 M2 R4 0.66	197	197	0.6	0%	1020	4337
MED J10 M2 R4 1.0	214	214	1.8	0%	1403	7803
MED J25 M3 R6 0.33	824	824	184.5	0%	4297	33,201
MED J25 M3 R6 0.66	548	548	116.3	0%	4249	43,286
MED J25 M3 R6 1.0	619	609	3600.9	2%	7730	123,864
MED J50 M5 R12 0.33	1718	1655	3601.3	4%	9240	93,821
MED J50 M5 R12 0.66	1704	1581	3603.1	7%	19,552	369,559
MED J50 M5 R12 1.0	2813	79	3601.7	97%	28,786	913,568
MED J75 M7 R18 0.33	2449	2249	3600.9	8%	21,707	349,682
MED J75 M7 R18 0.66	4398	107	3603.3	98%	38,683	1,248,299
MED J75 M7 R18 1.0	4897	114	3605.0	98%	67,784	3,214,873
MED J100 M10 R24 0.33	5683	155	3601.3	97%	35,072	740,634
MED J100 M10 R24 0.66	5583	165	3605.7	97%	73,046	3,204,091
MED J100 M10 R24 1.0	5461	144	3615.2	97%	107,061	6,875,867
HAR J10 M2 R4 0.33	432	432	1.4	0%	1935	6277
HAR J10 M2 R4 0.66	376	376	1.6	0%	1935	8239
HAR J10 M2 R4 1.0	406	406	8.8	0%	2720	15,157
HAR J25 M3 R6 0.33	1524	1524	348.7	0%	8182	63,302
HAR J25 M3 R6 0.66	1016	1016	328.2	0%	8225	84,006

Table 7-1 (continued)

Com Jobs Mach Res Den	Obj Funct	Best Bound	Sol Time (sec)	Gap (%)	Vars	Cons
HAR J25 M3 R6 1.0	1154	1146	3602.1	1%	14,610	233,934
HAR J50 M5 R12 0.33	3095	2940	3602.4	5%	17,546	178,486
HAR J50 M5 R12 0.66	3109	2702	3601.7	13%	37,414	708,309
HAR J50 M5 R12 1.0	2846	2001	3603.5	30%	55,270	1,756,920
HAR J75 M7 R18 0.33	4660	3974	3601.7	15%	41,696	672,674
HAR J75 M7 R18 0.66	7792	160	3605.2	98%	73,320	2,366,947
HAR J75 M7 R18 1.0	9639	160	3620.0	98%	130,768	6,207,290
HAR J100 M10 R24 0.33	9582	2670	3601.8	72%	66,446	1,403,993
HAR J100 M10 R24 0.66	10,442	239	3611.0	98%	138,488	6,080,351
HAR J100 M10 R24 1.0	—	—	—	—	—	—
Avg	2589	769	2334	37%	28,402	1,055,745
Std dev	2722	939	1704	45%	34,524	1,743,657

**Fig. 7-1.** IP Model Constraints vs Gap (%) Correlation.

more variables and constraints than the reduced. We believe, the Gap is not impacted due to factors such as multiple machines used and time horizon $|H|$ provided a tighter formulation potentially neutralizing the effect of process times. The average for the hard complexity includes a gap of 100% for the HAR|J100|M10|R24|1.0 combination which was not solved by the optimization model. The table below shows the average gap by number of jobs, density sharing (DS %) and by complexity. The complexity table has two additional columns to show the average number of variables and average number of constraints.

In order to better understand the behavior of this model with respect to run time, we selected the J50|M5|R12|0.66 scenario and ran it for 30, 60, 90 and 120 min for the three complexity levels (reduced, medium, hard). Table 7-3 presents the results. Prior to discussing the results, a striking observation that emerged from this experiment needs to be discussed. The initial models ran on a virtual server and the results obtained became slightly inferior as the model ran longer than 60 min. That is, Cplex found slightly better solutions for the 60-minute run than for the 90- and 120-minute runs. Upon investigation in technical forums and by analyzing solver logs, it was apparent this behavior has been reported out by other researchers and Cplex logs show Cplex inducing

Table 7-2

IP model gap % summary.

Jobs	Avg Gap %	DS (%)	Avg Gap%	Complexity	Avg Gap%	Avg Vars	Avg Const
10	0%	0.33	20.8%	reduced	39.9%	15,368	629,386
25	0%	0.66	40.6%	medium	40.3%	28,042	1,148,408
50	29%	1.0	54.1%	hard	35.3%	42,754	1,413,278
75	69%						
100	94%			Overall Gap	37%		

Table 7-3

J50|M5|R12|0.66 results for 30 to 120 min run time.

Complexity	Stop Criteria (min)	OF	BB	Sol Time (sec)	Gap (%)	Var	Con
Reduced	30	1047	785	1802	24.9%	10,622	200,233
	60	950	876	3601	7.8%		
	90	948	876	5400	7.6%		
	120	947	876	7201	7.5%		
Medium	30	1681	1579	1801	6.1%	19,552	369,559
	60	1681	1579	3601	6.1%		
	90	1671	1579	5401	5.5%		
	120	1671	1579	7201	5.5%		
Hard	30	3144	2404	1801	23.5%	37,414	708,309
	60	3099	2931	3601	5.4%		
	90	3099	2931	5401	5.4%		
	120	3091	2949	8121	4.6%		

Table 7-4

J50|M5|R12|0.66 results Cplex heuristic disabled 30 to 120 min run time.

Complexity	Stop Criteria (min)	Obj Funt	Best Bound	Sol Time (sec)	Gap (%)	Var	Con
Reduced	30	-1	872.7	1800	-	10,622	200,233
	60	989	876	3601	11.5%		
	90	989	876	5400	11.5%		
	120	964	876	7201	9.1%		
Medium	30	-1	1578	1801	-	19,552	369,559
	60	1738	1578	3601	9.2%		
	90	1734	1579	5401	9.0%		
	120	1734	1582	7201	8.7%		
Hard	30	-	2934.53	1800	-	37,414	708,309
	60	-	2946.69	3602	-		
	90	-	2947	5402	-		
	120	-	2947	7202	-		

what it seems to be a smart heuristic allowing the model to find a small improvement (less than 1%) minutes before model reached maximum run time. In order to fix this behavior, the model ran in a laptop to minimize the impact of a virtual server balancing computing loads and the results are depicted in Table 7-3.

It is evident that 30 min is not enough time to get a near optimal solution for the reduced and hard problem instances. However, the results are the same for the medium 30- and 60-minute instance. The most noticeable improvement is from 30 to 60 min, running the model for 90 and 120 min did not have a significant effect for this problem instance.

Additionally, the model ran on a laptop with 3 parameters disabled as suggested in technical forums in order to remove the unexpected behavior of the solver. The results presented in Table 7-4 were obtained by applying the following settings: MIP.Strategy.RINSHeur = -1; MIP.Strategy.FPHeur = -1, and MIP.Strategy.LBHeur = 0 which disabled Cplex built-in heuristics. The columns of table below present the objective function, best bound, solution time, Gap %, number of

variables and constraints grouped by complexity and maximum run time (stop criteria) given in minutes.

Results in Tables 7-3 and 7-4 show that Cplex was consistent in finding either the same or better result as the model ran longer. However, disabling Cplex heuristics is not an option as many scenarios did not find a feasible solution (“-”) and the results were significantly inferior.

In summary, running the model longer than 60-minutes may not be the right decision for practical problems that resemble the characteristics of the J50 size with 66% job-machine sharing density.

Next we present the results obtained by the proposed heuristic.

7.2. Experimental results heuristic for $Pm|s_{jk}, rt_j, aux| \sum C_j$

The heuristic results for the 45 problem instances are presented in Table 7-5. The objective function value was calculated by adding up the completion time for each job (total completion time). It is evident that

Table 7-5
Heuristic Results.

Com Jobs Mach Res Den	Min Obj Funct	Avg Obj Funct	Std Obj Funct	Avg Sol Time (sec)	Std Sol Time (sec)
RED J10 M2 R4 0.33	141	151	12	0.4	0.1
RED J10 M2 R4 0.66	137	146	7	0.5	0.1
RED J10 M2 R4 1.0	127	135	7	0.4	0.0
RED J25 M3 R6 0.33	556	577	28	1.4	0.1
RED J25 M3 R6 0.66	418	441	15	1.8	0.2
RED J25 M3 R6 1.0	427	440	8	2.8	0.5
RED J50 M5 R12 0.33	1400	1445	35	3.4	0.3
RED J50 M5 R12 0.66	1062	1175	91	4.9	0.4
RED J50 M5 R12 1.0	932	987	47	11.6	1.8
RED J75 M7 R18 0.33	1947	2010	51	6.7	0.5
RED J75 M7 R18 0.66	1452	1509	41	15.0	1.5
RED J75 M7 R18 1.0	1593	1660	34	29.6	6.4
RED J100 M10 R24 0.33	2366	2519	75	10.2	0.7
RED J100 M10 R24 0.66	2010	2155	79	28.3	5.6
RED J100 M10 R24 1.0	1930	1993	30	47.5	7.3
MED J10 M2 R4 0.33	263	274	16	0.4	0.1
MED J10 M2 R4 0.66	227	246	15	0.5	0.1
MED J10 M2 R4 1.0	244	250	5	0.5	0.1
MED J25 M3 R6 0.33	985	1060	75	1.4	0.2
MED J25 M3 R6 0.66	738	762	15	1.8	0.1
MED J25 M3 R6 1.0	721	752	28	3.6	0.4
MED J50 M5 R12 0.33	2457	2521	44	3.0	0.2
MED J50 M5 R12 0.66	2110	2209	104	5.8	0.7
MED J50 M5 R12 1.0	1651	1752	84	13.3	2.8
MED J75 M7 R18 0.33	3333	3435	57	6.0	0.3
MED J75 M7 R18 0.66	2723	2760	35	14.4	0.9
MED J75 M7 R18 1.0	2720	2878	158	22.9	1.8
MED J100 M10 R24 0.33	4183	4287	63	9.9	0.7
MED J100 M10 R24 0.66	3544	3667	71	29.9	7.7
MED J100 M10 R24 1.0	3267	3418	122	42.2	2.3
HAR J10 M2 R4 0.33	497	516	30	0.5	0.2
HAR J10 M2 R4 0.66	437	463	19	0.4	0.1
HAR J10 M2 R4 1.0	477	481	4	0.5	0.1
HAR J25 M3 R6 0.33	1865	1954	116	1.7	0.2
HAR J25 M3 R6 0.66	1336	1509	101	3.6	1.3
HAR J25 M3 R6 1.0	1349	1387	23	3.9	0.4
HAR J50 M5 R12 0.33	4612	4665	53	4.0	0.4
HAR J50 M5 R12 0.66	3994	4073	51	5.9	0.2

Table 7-5 (continued)

Com Jobs Mach Res Den	Min Obj Funct	Avg Obj Funct	Std Obj Funct	Avg Sol Time (sec)	Std Sol Time (sec)
HAR J50 M5 R12 1.0	3295	3334	37	12.2	2.4
HAR J75 M7 R18 0.33	6244	6452	136	7.4	0.7
HAR J75 M7 R18 0.66	4757	5032	146	15.2	1.1
HAR J75 M7 R18 1.0	5187	5338	110	24.3	1.7
HAR J100 M10 R24 0.33	7590	7689	63	12.0	1.5
HAR J100 M10 R24 0.66	6213	6780	232	25.5	4.2
HAR J100 M10 R24 1.0	6039	6271	143	42.2	3.3
Avg	2212	2301	60	11	1
Std dev	1938	2006	–	12	–

the heuristic run times are fast with a range of 0.4 s for the “RED|J10|M2|R4|D0.33” and 47 s for the RED|J100|M10|R24|1.0 scenario. The table below shows the minimum objective found by the heuristic, the average objective found for the 32 combinations, the standard deviation of the objective function. The last two columns provide the average solution time and the standard deviation (in seconds).

It is evident from Table 7-5 that the average objective function provided by the heuristic (2212) is lower than the average obj function provided by the IP model (2589). Section 7.4 shows the breakdown by scenario for the IP model, heuristic and the hybrid (heuristic + IP model). A total of 32 sorting variations were run per problem instance with the best solution given to the IP model in order to prime it with an initial feasible solution.

A short summary characterizing the sorting results are provided for small, medium and large problem sizes. It is evident from the results that for the problem size instances with 10 jobs all instances achieved the best results when post initialization sort was: last_completion_time (desc), job_ready_time (desc), family (desc). For 4 out of 9 combinations for job size = 10 the best results were obtained when the initialization sorting logic was the following: m_score (asc), process time (desc), family (asc), mr_score (asc). For problem instances with 25 jobs 4 out of 9 scenarios obtained the best results when the initialization sort logic was the following: m_score (asc), process time (asc), family (desc), mr_score (desc). Similarly, for J50 problem instances 6 out of 9 scenarios obtained best results when initialization sort logic was -> m_score (asc), process time (asc), family (desc), mr_score (desc). For problem instances with 75 and 100 jobs no sorting logic dominated the best results. In summary, the post initialization sorting logic had more effect on the smaller problem instances than the larger ones.

7.3. Experimental results hybrid (IP/Heuristic) for $Pm|s_{jk}, rt_j, aux| \sum C_j$

Table 7-6 presents the results for the hybrid model consisting of the heuristic feasible solution provided to the IP model by the heuristic. As previously stated, the heuristic solutions were provided to the IP model as “MIP start” in order to reduce time horizon and to provide the IP model with an initial solution which could be further improved. Reducing the time-indexed IP formulation horizon results in a smaller solution space which is likely to provide better results in less time. It is also important to highlight that providing an initial solution from the heuristic reduced time horizon by ~ 50% when H formulation was used instead of the H’ presented in section 4.

The feasible solution found by the heuristic for problem instance “HAR|J100|M10|R24|D1.0” was not further improved by the hybrid model in one hour due to the large number of constraints generated (9,458,057).

It is evident From Table 7-7 that the hybrid model Gap % follows a

Table 7-6Hybrid IP/Heuristic Results for $Pm|s_{jk}, rt_j, aux| \sum C_j$.

Com Jobs Mach Res Den	Obj Funct	Best Bound	Sol Time (sec)	Gap (%)	Vars	Cons
RED J10 M2 R4 0.33	128	128	0.5	0%	542	1740
RED J10 M2 R4 0.66	110	110	0.3	0%	556	2362
RED J10 M2 R4 1.0	118	118	0.5	0%	756	4207
RED J25 M3 R6 0.33	464	464	43.3	0%	2072	15,715
RED J25 M3 R6 0.66	322	322	29.6	0%	2409	24,466
RED J25 M3 R6 1.0	341	341	608.5	0%	4064	64,915
RED J50 M5 R12 0.33	992	957	3601.8	4%	7406	77,776
RED J50 M5 R12 0.66	1062	63	3600.7	94%	9877	184,813
RED J50 M5 R12 1.0	932	62	3601.3	93%	15,794	500,112
RED J75 M7 R18 0.33	1454	1293	3601.9	11%	11,891	191,405
RED J75 M7 R18 0.66	1452	85	3602.2	94%	21,361	689,285
RED J75 M7 R18 1.0	1593	88	3605.2	94%	31,565	1,472,594
RED J100 M10 R24 0.33	2366	118	3601.9	95%	19,385	409,063
RED J100 M10 R24 0.66	2010	125	3605.3	94%	40,324	1,769,745
RED J100 M10 R24 1.0	1930	122	3611.4	94%	55,085	3,507,813
MED J10 M2 R4 0.33	232	232	0.5	0%	942	3010
MED J10 M2 R4 0.66	197	197	0.4	0%	972	4085
MED J10 M2 R4 1.0	214	214	1.7	0%	1243	6715
MED J25 M3 R6 0.33	824	824	231.5	0%	4157	31,997
MED J25 M3 R6 0.66	548	548	77.6	0%	3955	39,872
MED J25 M3 R6 1.0	616	616	3045.4	0%	6305	98,024
MED J50 M5 R12 0.33	1724	1655	3601.8	4%	12,280	128,173
MED J50 M5 R12 0.66	1696	1578	3603.8	7%	17,764	332,551
MED J50 M5 R12 1.0	1492	1202	3602.3	19%	25,786	808,508
MED J75 M7 R18 0.33	2477	2250	3602.2	9%	21,707	349,682
MED J75 M7 R18 0.66	2723	107	3603.9	96%	35,434	1,132,478
MED J75 M7 R18 1.0	2720	114	3607.9	96%	56,759	2,639,536
MED J100 M10 R24 0.33	3861	2189	3605.4	43%	35,072	740,634
MED J100 M10 R24 0.66	3544	165	3610.1	95%	71,000	3,104,173
MED J100 M10 R24 1.0	3267	144	3622.1	96%	86,061	5,375,249
HAR J10 M2 R4 0.33	432	432	1.2	0%	1647	5197
HAR J10 M2 R4 0.66	376	376	0.9	0%	1647	6727
HAR J10 M2 R4 1.0	406	406	3.3	0%	2120	11,077
HAR J25 M3 R6 0.33	1524	1524	390.5	0%	6467	48,553
HAR J25 M3 R6 0.66	1016	1016	95.0	0%	5873	56,694
HAR J25 M3 R6 1.0	1154	1154	3124.5	0%	9810	146,894

Table 7-6 (continued)

Com Jobs Mach Res Den	Obj Funct	Best Bound	Sol Time (sec)	Gap (%)	Vars	Cons
HAR J50 M5 R12 0.33	3097	2936	3604.1	5%	20,106	207,414
HAR J50 M5 R12 0.66	3146	2942	3603.5	6%	29,517	544,857
HAR J50 M5 R12 1.0	2817	2191	3605.7	22%	41,020	1,257,885
HAR J75 M7 R18 0.33	4618	3972	3603.5	14%	40,786	656,449
HAR J75 M7 R18 0.66	4664	2497	3609.4	46%	60,324	1,903,663
HAR J75 M7 R18 1.0	5187	2371	3628.1	54%	93,493	4,262,103
HAR J100 M10 R24 0.33	7575	3197	3606.9	58%	63,176	1,327,463
HAR J100 M10 R24 0.66	6213	239	3634.6	96%	111,208	4,748,111
HAR J100 M10 R24 1.0	6039	198	3645.5	97%	153,025	9,458,057
Avg	1993	931	2335	0	27,617	1,074,485
Std dev	1838	1039	1691	0	33,552	1,863,529

Table 7-7

Hybrid model gap% summary.

Jobs	Avg Gap %	DS (%)	Avg Gap%	Complexity	Avg Gap%
10	0%	0.33	19.7%	reduced	44.9%
25	0%	0.66	42.0%	medium	34.6%
50	28%	1.0	44.4%	hard	26.6%
75	57%				
100	91%			Total Gap 34%	

similar pattern as the optimization model where the gap grows as there are more jobs and the sharing density grows. The process times used to simulate the complexity do not make the gap worse since we used H' making the model tighter.

Optimal solutions were found for 18 out of the 45 scenarios in less than one hour for all the combinations of 10/25 job sizes. A Gap of 10% or less was found for 24 out of the 45 scenarios ranging from 10 jobs up to 75 jobs (i.e. MED|J75|M7|R18|0.33). Optimal solutions were found for 15 out of the 45 scenarios for the 10/25 job size instances in less than 600 s (10 min).

7.4. Best integer solution comparison

Table 7-8 shows the best integer solution found for smaller size problem instances and the comparison among them. Optimal solutions were obtained in one hour or less for all the scenarios executed by the hybrid model. The optimization IP model alone shows a Gap of 2% and 1% for scenarios "MED|J25|M3|R6|1.0" and "HAR|J25|M3|R6|1.0" when stopped after one hour. The best integer solution % delta was calculated as follows:

$$1 - \frac{Opt}{Heur}, 1 - \frac{Hybr}{Heur}, 1 - \frac{Hybr}{Opt}, \text{ respectively.}$$

In summary, the heuristic does not add a lot of value for the small scenarios since the IP model is faster and obtains the same results.

Table 7-9 shows the best integer solution found for medium size problem instances and the comparison among them. As expected, the hybrid model outperformed the IP model when stopped after one hour except for three instances: RED|J50|M5|R12|0.66, MED|J75|M7|R18|0.33, HAR|J50|M5|R12|0.66. The IP model found a better solution due to the reduced time horizon provided by H' formulation and the heuristic initial solution not being as efficient. Additionally, four instances *|J50|M5|R12|0.33 and MED|J75|M7|R18|0.33 did not accept the initial solution of the heuristic since the H' formulation was more efficient than the Cmax provided by the heuristic. Hence, the H'

Table 7-8

Best Solution Small Scenarios (10/25 Jobs).

Com Jobs Mach Res Den	Best Integer Solution			Solution Time (sec)			Best integer solution % delta		
	IP	Heur	Hybr	IP	Heur	Hybr	IP vs Heur	Hybr vs Heu	Hybr vs IP
RED J10 M2 R4 0.33	128	141	128	1	0	0	9%	9%	0%
RED J10 M2 R4 0.66	110	137	110	0	0	0	20%	20%	0%
RED J10 M2 R4 1.0	118	127	118	1	0	0	7%	7%	0%
RED J25 M3 R6 0.33	464	556	464	46	1	43	17%	17%	0%
RED J25 M3 R6 0.66	322	418	322	43	2	30	23%	23%	0%
RED J25 M3 R6 1.0	341	427	341	693	3	609	20%	20%	0%
MED J10 M2 R4 0.33	232	263	232	3	0	1	12%	12%	0%
MED J10 M2 R4 0.66	197	227	197	1	0	0	13%	13%	0%
MED J10 M2 R4 1.0	214	244	214	2	1	2	12%	12%	0%
MED J25 M3 R6 0.33	824	985	824	185	1	232	16%	16%	0%
MED J25 M3 R6 0.66	548	738	548	116	2	78	26%	26%	0%
MED J25 M3 R6 1.0	619	721	616	3601	4	3045	14%	15%	0%
HAR J10 M2 R4 0.33	432	497	432	1	1	1	13%	13%	0%
HAR J10 M2 R4 0.66	376	437	376	2	0	1	14%	14%	0%
HAR J10 M2 R4 1.0	406	477	406	9	1	3	15%	15%	0%
HAR J25 M3 R6 0.33	1524	1865	1524	349	2	391	18%	18%	0%
HAR J25 M3 R6 0.66	1016	1336	1016	328	4	95	24%	24%	0%
HAR J25 M3 R6 1.0	1154	1349	1154	3602	4	3124	14%	14%	0%
Average							16%	16%	0%
Std Dev							5%	5%	0%

Table 7-9

Best Solution Medium Scenarios (50/75 Jobs).

Com Jobs Mach Res Den	Best integer solution			Solution time (sec)			Best integer solution % delta		
	IP	Heur	Hybr	IP	Heur	Hybr	IP vs Heur	Hybr vs Heu	Hybr vs IP
RED J50 M5 R12 0.33	991	1400	992	3601	3	3602	29%	29%	0%
RED J50 M5 R12 0.66	937	1062	1062	3602	5	3601	12%	0%	-13%
RED J50 M5 R12 1.0	1660	932	932	3601	12	3601	-78%	0%	44%
RED J75 M7 R18 0.33	1451	1947	1454	3602	7	3602	25%	25%	0%
RED J75 M7 R18 0.66	2231	1452	1452	3602	15	3602	-54%	0%	35%
RED J75 M7 R18 1.0	2545	1593	1593	3603	30	3605	-60%	0%	37%
MED J50 M5 R12 0.33	1718	2457	1724	3601	3	3602	30%	30%	0%
MED J50 M5 R12 0.66	1704	2110	1696	3603	6	3604	19%	20%	0%
MED J50 M5 R12 1.0	2813	1651	1492	3602	13	3602	-70%	10%	47%
MED J75 M7 R18 0.33	2449	3333	2477	3601	6	3602	27%	26%	-1%
MED J75 M7 R18 0.66	4398	2723	2723	3603	14	3604	-62%	0%	38%
MED J75 M7 R18 1.0	4897	2720	2720	3605	23	3608	-80%	0%	44%
HAR J50 M5 R12 0.33	3095	4612	3097	3602	4	3604	33%	33%	0%
HAR J50 M5 R12 0.66	3109	3994	3146	3602	6	3604	22%	21%	-1%
HAR J50 M5 R12 1.0	2846	3295	2817	3603	12	3606	14%	15%	1%
HAR J75 M7 R18 0.33	4660	6244	4618	3602	7	3604	25%	26%	1%
HAR J75 M7 R18 0.66	7792	4757	4664	3605	15	3609	-64%	2%	40%
HAR J75 M7 R18 1.0	9639	5187	5187	3620	24	3628	-86%	0%	46%
Average							-18%	13%	18%
Std Dev							48%	13%	22%

formulation was modified for these instances from $\Theta = 2$ to $\Theta = 3$ in order to extend the time horizon and accept the initial solution from the heuristic. The average for the best integer solution delta (%) is provided at the bottom of the table. The results show the heuristic adds more value for the medium size scenarios than for small scenarios.

Table 7-10 shows that for the large size (100 jobs) the heuristic initial solution was further improved by the hybrid model for only two out of 9 scenarios. That shows the heuristic becomes more attractive as the number of jobs increases above 100 jobs. In summary, the average best integer solution of the optimization IP model vs heuristic is -50% showing the heuristic adds more value as feasible solutions were found and the IP formulation failed to find efficient solutions within one hour.

It is evident From Table 7-10 that the solutions found by the hybrid model outperformed the IP model for the large instances when a stop

limit of one hour is used (i.e. optimization IP model has an average Gap of $\sim 33\%$ if the hybrid solution is used as the lower bound).

Overall, we believe these results should provide insight to industry practitioners to estimate solution time feasibility for real world complex problems requiring multiple setups and including ready times. These results provide insight to how the proposed time-indexed IP model, heuristic and hybrid models behaved for the 45 different problem instances. We believe the proposed models should allow practitioners to find optimal solutions for small and medium problem instances. This study also provides insight on how the time-index IP formulation can be applied to larger models if practitioners can transform 1 unit of time into y units of time per period t where $y > 1$. For example, if processing times take anywhere from 100 to 150 min, these process times can be divided by 5 min to find a reduced time horizon which in turns reduces the

Table 7-10
Best solution large scenario (100 Jobs).

Com Jobs Mach Res Den	Best integer solution			Solution time (sec)			Best integer solution % delta		
	IP	Heur	Hybr	IP	Heur	Hybr	IP vs Heur	Hybr vs Heu	Hybr vs IP
RED J100 M10 R24 0.33	3072	2366	2366	3602	10	3602	−30%	0%	23%
RED J100 M10 R24 0.66	3087	2010	2010	3604	28	3605	−54%	0%	35%
RED J100 M10 R24 1.0	3063	1930	1930	3610	48	3611	−59%	0%	37%
MED J100 M10 R24 0.33	5683	4183	3861	3601	10	3605	−36%	8%	32%
MED J100 M10 R24 0.66	5583	3544	3544	3606	30	3610	−58%	0%	37%
MED J100 M10 R24 1.0	5461	3267	3267	3615	42	3622	−67%	0%	40%
HAR J100 M10 R24 0.33	9582	7590	7575	3602	12	3607	−26%	0%	21%
HAR J100 M10 R24 0.66	10,442	6213	6213	3611	26	3635	−68%	0%	40%
HAR J100 M10 R24 1.0	–	6039	6039	–	42	3645	–	0%	–
Average							−50%	1%	33%
Std Dev							17%	3%	7%

computational complexity of the model. That is, this transformation allows the modeler to control computational complexity by keeping the time horizon from growing extremely large; hence, preventing the model from becoming increasingly complex. Lastly, practitioners may also modify time-indexed IP model constraint (Eq. (3)) from an “=” sign to a “≤” sign as shown below in order to keep the model from growing extremely large and complex.

$$\sum_{m \in M} \sum_{r \in R} \sum_{t=rt_j}^{H-p_j+1} x_{jmrt} \leq 1 \forall j \in J$$

Additionally, a new constraint must be added in order to force at least n lots to be scheduled where n is approximately 50 jobs in order to obtain near optimal solutions in relatively short time by controlling the time horizon.

8. Conclusions and future work

This study proposes an exact time-indexed IP model and a heuristic algorithm (hybrid) for parallel machine scheduling with dual resources, ready times, and multiple setups. The proposed IP $Pm|s_{jk}, rt_j, aux| \sum C_j$ model a time-indexed formulation proposed by (Sousa and Wolsey, 1992). The original model was expanded to include multiple machines, auxiliary resources, and sequence-dependent setups (i.e. resource and resource travel time between machines). The heuristic was developed with the objective to find fast, feasible solutions that can be provided to prime the time-indexed IP model in order to reduce the time horizon and provide an initial solution that could be further improved.

The hybrid model (IP model with initial solution given from heuristic) found optimal solutions for 18 out of the 45 scenarios in less than one hour for all of the problem instances of 10/25 jobs. A Gap of 10% or less was found for 24 out of the 45 scenarios ranging from 10 jobs up to 75 jobs (i.e. MED|J75|M7|R18|0.33). Optimal solutions were found for 15 out of the 45 scenarios for the 10/25 job size instances in less than 600 s (10 min). The heuristic initial solutions for the 45 problem instances were improved by the hybrid model by 2% to 33%. As expected, the “hard” scenario with 100 jobs was not solved in less than one hour due to the excessive number of constraints (9,458,057) which are mainly generated by equations (3) and (4). These equations prevent overlap among jobs, machines, and resources for each period and include both setups (i.e. resources and resource travel time). To our knowledge, the proposed IP formulation has not been published before nor has solutions for small (10/25 Jobs) size problems been solved to optimality in less than 10 min with this formulation.

In the future, we plan to further improve the heuristic that is currently providing “good” feasible solutions in a relative short amount of time by testing more variations beyond the 32 combinations per instance. We also plan to incorporate multiple objectives (multi-

objective) allowing practitioners to use this formulation for real-world problems where there is a need to model machines with resources and setups. If setups were omitted, the run time would be reduced significantly as the number of variables and constraints would be reduced. We believe it would be interesting to solve the time-index IP formulation using a rolling horizon approach to maintain the time horizon short and efficient.

In conclusion, a hybrid IP model with heuristic can add value to practitioners looking to solve practical scheduling systems with parallel machines, dual resources and sequence-dependent setups. It is extremely important to control problem instance size and time horizon to obtain near-optimal solutions within acceptable run times. The proposed heuristic primes with the IP model to reduce the space search by reducing the time horizon and provide an initial solution to the optimization model.

CRedit authorship contribution statement

L. Munoz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration. **J.R. Villalobos:** Conceptualization, Methodology, Formal analysis, Writing – review & editing, Visualization, Supervision, Project administration. **J. Fowler:** Conceptualization, Methodology, Formal analysis, Writing – review & editing, Visualization, Supervision, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Abeledo, H., Fukasawa, R., Pessoa, A., & Uchoa, E. (2010). The time dependent traveling salesman problem: Polyhedra and branch-cut-and-price algorithm. *Ln: Festa P (Ed) Experimental Algorithms, Lecture Notes in Computer Science, Vol 6049, Springer Berlin Heidelberg*, Pp202-213.
- Akcal, E., Nemoto, K., Uzsoy, R., 2001. Cycle-time improvements for photolithography process in semiconductor manufacturing. *IEEE Trans. Semicond. Manuf.* 14 (1), 48–56. <https://doi.org/10.1109/66.909654>.
- Allahverdi, A., Gupta, J., Aldowaisan, T., 1999. A survey of scheduling research involving setup considerations. *OMEGA – Int. J. Manage. Sci.* 27, 219–239.
- Allahverdi, A., 2015. The third comprehensive survey on scheduling problems with setup times/costs. *Eur. J. Oper. Res.* 246 (2), 345–378. <https://doi.org/10.1016/j.ejor.2015.04.004>.
- Allahverdi, A., Ng, C.T., Cheng, T.C.E., Kovalyov, M.Y., 2008. A survey of scheduling problems with setup times or costs. *Eur. J. Oper. Res.* 187 (3), 985–1032. <https://doi.org/10.1016/j.ejor.2006.06.060>.
- Avella, P., Boccia, M., D'Auria, B., 2005. Near-optimal solutions of large-scale single machine scheduling problems. *INFORMS J. Comput.* 17 (2), 183–191.

- Battarra, M., Pessoa, A., Subramanian, A., Uchoa, E. (2013). *Exact algorithms for the traveling salesman problem with draft limits*. Optimization Online Repositories.
- Behnamian, J., Zandieh, M., Fatemi Ghomi, S.M.T., 2009. Parallel-machine scheduling problems with sequence-dependent setup times using an ACO, SA and VNS hybrid algorithm. *Expert Syst. Appl.* 36 (6), 9637–9644. <https://doi.org/10.1016/j.eswa.2008.10.007>.
- Bigras, L., Gamache, M., Savard, G., 2008. The time dependent traveling salesman problem and single machine scheduling problems with sequence dependent setup times. *Discrete Optimization* 5 (4), 685–699.
- Blazewicz, J., Dror, M., Weglarz, J., 1991. Mathematical programming formulations for machine scheduling: a survey. *Eur. J. Oper. Res.* 51 (3), 283–300.
- Chudak, F., Hochbaum, D., 1999. A half-integral linear programming relaxation for scheduling precedence-constrained jobs on a single machine. *Oper. Res. Lett.* 25 (5), 199–204.
- Díaz, S.L.M., Fowler, J.W., Pfund, M.E., Mackulak, G.T., Hickie, M., 2005. Evaluating the impacts of reticle requirements in semiconductor wafer fabrication. *IEEE Trans. Semicond. Manuf.* 18 (4), 622–632.
- Dyer, M., Wosley, L., 1990. Formulating the single machine sequencing problem with release dates as a mixed integer program. *Discrete Appl. Math.* 26 (2–3), 255–270.
- Fan, B., Tang, G. (2006). A column generation for a parallel machine scheduling with sequence-dependent setup times. *Tongji Daxue Xuebao/J. Tongji Univ.* 34(5), 680–683 +693.
- Ham, A., 2018. Scheduling of dual resource constrained lithography production: using CP and MIP/CP. *IEEE Trans. Semicond. Manuf.* 31 (1), 52–61.
- Ham, A.M., Cho, M., 2015. A practical two-phase approach to scheduling of photolithography production. *IEEE Trans. Semicond. Manuf.* 28 (3), 367–373. <https://doi.org/10.1109/TSM.2015.2451512>.
- Hopp, W., Spearman, M. (2011). *Factory physics* (Waveland Press (ed.)). Boston :Irwin/McGraw-Hill.
- Hou, Z.-L., Guo, X.-P., 2013. In: *Proceedings of 2012 3rd International Asia Conference on Industrial Engineering and Management Innovation (IEMI2012)*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 801–811.
- Hu, D., Yao, Z., 2010. Genetic algorithms for parallel machine scheduling with setup times. Paper Presented at the 2nd International Conference on Information Science and Engineering.
- Hunter, J., Delp, D., Collins, D., Si, J., 2002. Understanding a semiconductor process using a full-scale model. *IEEE Trans. Semicond. Manuf.* 15 (2), 285–289.
- Khowala, K., Keha, A., Fowler, J., 2005. A comparison of different formulations for the non-preemptive single machine total weighted tardiness scheduling problem. *The Second Multidisciplinary International Conference on Scheduling: Theory & Application (MISTA)*.
- Lasserre, J., Queyranne, M. (1992). Generic scheduling polyhedral and a new mixed integer formulation for single-machine scheduling. *Ln: Pittsburg: Carnegie Mellon University*.
- Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G., Shmoys, D.B., 1993. Sequencing and scheduling: algorithms and complexity. *Handb. Oper. Res. Manage. Sci.* 4 (C), 445–522. [https://doi.org/10.1016/S0927-0507\(05\)80189-6](https://doi.org/10.1016/S0927-0507(05)80189-6).
- Manne, A.S., 1960. On the job-shop scheduling problem. *Oper. Res.* 8 (2), 159–224.
- Nogueira, T., Carvalho, C., Ravetti, M., Souza, M., 2019. Analysis of mixed integer programming formulations for single machine scheduling problems with sequence dependent setup times and release dates. *Pesquisa Operacional* 39 (1), 109–154. http://www.optimization-online.org/DB_HTML/2014/07/4442.html.
- Pessoa, A., Uchoa, E., Poggi, M., Rosiane, D.A., 2010. Exact algorithm over an arc-time-indexed formulation for parallel machine scheduling problems. *Math. Program. Comput.* 2, 259–290. <https://doi.org/10.1007/s12532-010-0019-z>.
- Pinedo, M.L., 2016. *Scheduling Theory, Algorithms, and Systems*, (5th ed.). Springer.
- Potts, C. (1980). An algorithm for the single-machine sequencing problem with precedence constraints. *Lecture Notes: Combinatorial Optimization II, Springer, Pp 78–87*.
- Queyran, M., 1993. Structure of a simple scheduling polyhedron. *Math. Program.* 58, 263–285.
- Queyranne, M., Wang, Y., 1991. Single-machine scheduling polyhedra with precedence constraints. *Math. Oper. Res.* 16 (1), 1–20.
- Rocha, P.L., Ravetti, M.G., Mateus, G.R., Pardalos, P.M., 2008. Exact algorithms for a scheduling problem with unrelated parallel machines and sequence and machine-dependent setup times. *Comput. Oper. Res.* 35 (4), 1250–1264. <https://doi.org/10.1016/j.cor.2006.07.015>.
- Shahvari, O., Logendran, R., 2017. A bi-objective batch processing problem with dual-resources on unrelated-parallel machines. *Appl. Soft Comput. J.* 61, 174–192. <https://doi.org/10.1016/j.asoc.2017.08.014>.
- Sousa, J.P., Wosley, L.A., 1992. A time indexed formulation of non-preemptive single machine scheduling problems. *Math. Program.* 54, 353–367. <https://doi.org/10.1007/BF01586059>.
- Toksari, M.D., Güner, E., 2010. Parallel machine scheduling problem to minimize the earliness/tardiness costs with learning effect and deteriorating jobs. *J. Intell. Manuf.* 21 (6), 843–851. <https://doi.org/10.1007/s10845-009-0260-3>.
- Unlu, Y., Mason, S.J., 2010. Computers & Industrial Engineering Evaluation of mixed integer programming formulations for non-preemptive parallel machine scheduling problems q. *Comput. Ind. Eng.* 58 (4), 785–800. <https://doi.org/10.1016/j.cie.2010.02.012>.
- Wagner, H.M., 1959. An integer linear-programming model for machine scheduling. *Nav. Res. Logist.* 6 (2), 131–140.
- Yan, B., Chen, H., Luh, P., Wang, S., Chang, J., 2013. Litho machine scheduling with convex hull analyses. *IEEE Trans. Autom. Sci. Eng.* 10 (4), 928–937. <https://doi.org/10.1109/TASE.2013.2277812>.
- Yan, B., Chen, H., Luh, P., Wang, S., Chang, J., 2012. Optimization-based Litho Machine Scheduling with Load Balancing and Reticle Expiration. In: *8th IEEE International Conference on Automation Science and Engineering*. <https://doi.org/10.1109/CoASE.2012.6386493>.
- Yan, B., Chen, H., Luh, P., Wang, S., Chang, J., 2011. Optimization-based litho machine scheduling with multiple reticles and setups. *IEEE Int. Conf. Automation Sci. Eng.* 2011, 114–119. <https://doi.org/10.1109/CASE.2011.6042516>.